

Check point 1

Suppose we have a 20% dropout rate for 100 neurons

1.1) About how many neurons are active on any training step?

80

1.2) Does the set of active neurons stay the same throughout training?

No, we are randomly zeroing out 20% of neurons

Does this mean every neuron has a 20% chance of being knocked out, or do we pick 20 neurons to be knocked out? I assume the former, as that's likely easier to parallelize, and for a large number of neurons, there won't be much of a difference.

Checkpoint 2

Suppose a neuron that your model relied upon is randomly removed during one training batch.

What pressure does that create for the rest of the network?

It encourages neurons to not be completely dependent

on any particular neuron, but instead to accept inputs across a range of neurons from the previous layer.

Checkpoint 3

Suppose we use 50% dropout.

A neuron outputs 8, and survives this training step.

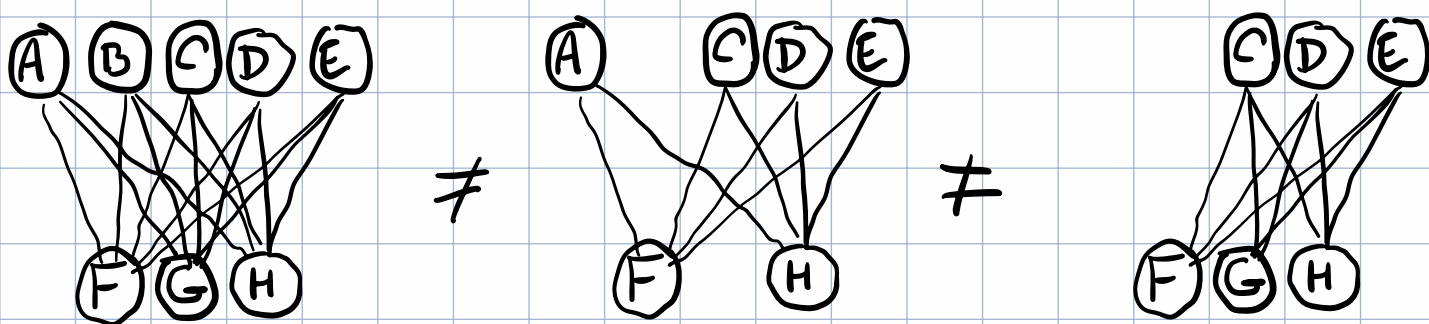
Using inverted dropout, what value is passed to the next layer?

$$8 / .5 = 16$$

Exercises

1) Why does dropout reduce overfitting, even though the network architecture never changes.

The effective network does change.



The network learns to work with missing data, so the

Values aren't going to be the same every time.

2) Why is dropout only used in training?

Dropout does reduce the accuracy of the output of the system, but during training all that matters is the relative performance.

This is similar to an athlete running with weights during training, but not during the real race. Then when it really matters it's like they are lighter, since they've gotten used to that weight.

3) Complete checkpoints 1, 2, & 3

Done

4) Suppose you raise dropout from 20% to 80%. What trade-off do you expect during training and unseen data?

The quality of output will decline in training. Raising it to 80% might make it impossible for the system to learn the patterns in the data.

If 80% doesn't constrain the system too much, and it is still able to learn, then the system should be pretty robust to surprising data

when evaluated at test time.

5) Compare L2 regularization, early stopping, and dropout. What problem does each address, and how are they similar?

L2 regularization penalizes solutions that are overly specific.

Early stopping stops training before the system learns to just memorize the test data, along with the noise in it.

Dropout stops the system from overly relying on specific nodes.

All three are used to discourage over-fitting, but at different levels of the process.

L2 regularization works at the function emulation level.

Early Stopping works at the training method level.

Dropout works at the network level.